Problem Set 3: Part I

Problem 1: Printing the odd values in a list of integers

1-1)

```
public static void printOddsRecur(IntNode first) {
        // return if empty or we reached end of list
        if (first == null) return;

        // use bitwise and to check even/odd, print odds only
        if ((first.val & 1) == 1) { System.out.println(first.val); }

        // call on the next node in linked list
        printOddsRecur(first.next);

}
```

1-2)

```
public static void printOddsIter(IntNode first) {
      // check if null, check if odd, then step forward
        while (first != null) {
            if ((first.val & 1) == 1) {
                System.out.println(first.val);
            }
            first = first.next;
        }

}
```

Problem 2: Choosing an appropriate list implementation

2-1) ArrayList or LLList?
explanation:
Venue:
LLList - with the caveat that we both keep a reference to the last node and also have an iterator. With an iterator, printing the contents of either list would be O(n) time. If we kept a reference to the last node, adding an item to the end of the list would be O(1) in both cases. These are our two main use cases. The only difference is that the number of items varies slightly month over month, if we unexpectedly get a packed month, ArrayList could either a) run out of space or b) require us to use significantly more memory than an LLList on average. LLList wins by memory efficiency.

2-2) ArrayList or LLList?
explanation:
Marathon runners:
ArrayList - we'll need to constantly get the record, which is O(1) time in an array and O(n) on average in LLList. Changing the list has efficiency trade-offs for both, but we don't expect many changes. The main use case will be to getItem a random index from the list, where ArrayList wins, with or without an iterator.

2-3) ArrayList or LLList?
explanation:
LLList - because the number of registrants vary greatly, we would want a dynamic sized list. In terms of adding students, with a reference to the last node, this is O(1) time in both cases. We need to regularly display the list from most recently added (head of list) - this is O(n) time for an arraylist, and potentially O(n^2) time for a LinkedList, though only O(n) with an iterator. So displaying the list is O(n) in both cases. We choose LLList because of the consideration for the changing list sizes.

Problem 3: Analyzing and improving an algorithm

3-1) Without a reference to the last node, this is O(n^2). Adding the ith item (the ith loop), we need addItem to scaled, which takes i steps to walk down the linked list.

With a reference to the last node, this would be O(n) time, as adding an item to the end of the list is now constant time

getItem is constant time from an array, as is multiplication (I think).

3-2) Supposing we don't have a reference to the last item in the linked list, adding an item to the BEGINNING of a linked list is constant time, so we just move through the list backwards.

```
public static LLList scale(int factor, ArrayList vals) {

     LLList scaled = new LLList();

     int listLen = vals.length();

     for (int i = listLen - 1; i > -1; i = i-1) {
         int val = (Integer)vals.getItem(i);
         scaled.addItem(val * factor, 0);
         // add to beginning of list always
         }

     return scaled;

 }
```

3-3) The runtime of the improved algorithm is O(n). The functions inside the for loop run in constant time (getItem is constant in an array, and adding to the beginning of an LLList is constant time). Therefore, we just run the for-loop n times, for a total of n * constant runtime, which is O(n).

Problem 4: Working with stacks and queues

4-1)

```
   public static boolean containsStack(Stack<Object> stack, Object item) {

        // use LL as we don't know how big the original stack is
        LLStack<Object> orig = new LLStack<Object>();

        Object curr;
        if (stack == null || item == null || stack.isEmpty()) return false;

        // special case so we don't have to pull out ALL items
        // pre-assign curr so we can use it in our next if-statement
        curr = stack.pop();
        orig.push(curr);

        //  pull items out until we (potentially) hit the correct one
        while (!stack.isEmpty() && !item.equals(curr)) {
            curr = stack.pop();
            orig.push(curr);
        }

        // put everything back that we took out
        while (!orig.isEmpty()) {
            stack.push(orig.pop());
        }

        return (item.equals(curr));
    }
```

4-2)

```
   public static boolean containsQueue(Queue<Object> queue, Object item) {

        LLQueue<Object> orig = new LLQueue<Object>(); // don't know how long
queue will be

        // same principle as last time
        if (queue == null || item == null || queue.isEmpty()) return false;

        boolean rvalue = false;
        Object curr;


        // here, we need to pull everything out to maintain the same order
        while (!queue.isEmpty()) {
            curr = queue.remove();
            orig.insert(curr);
            if (item.equals(curr)) rvalue = true;
        }
        // put everything back
        while (!orig.isEmpty()) queue.insert(orig.remove());

        return rvalue;
```

```
}
```